

Questions«Vrai ou Faux»

- **Q1:**
- Si une méthode d'analyse est bien appliquée, le résultat devrait toujours être sensiblement pareil entre deux analyses sur un même sujet.

Questions«Vrai ou Faux»

- **R1:**
- C'est faux parce que l'analyse fait appel à divers talents: expérience, connaissance du domaine d'expertise, talent artistique...
- Il pourra y avoir de nombreux résultats valables. Notons toutefois qu'en général, on doit toujours rechercher le design le plus harmonieux et le plus simple possible car c'est plus facile à comprendre et à maintenir.

Questions«Vrai ou Faux»

- **Q2:**
- Autant que possible, les attributs d'une classe devraient être déclarés privés et méthodes déclarées publiques.

Questions«Vrai ou Faux»

- **R2:**

C'est **vrai** parce qu'il faut résoudre le problème de couplage des données.

On maintient les données privées (attributs) et on force le programmeur utilisateur de la classe à passer par l'interface publique de la classe.

Questions«Vrai ou Faux»

- **Q3:**
- On peut facilement intégrer les technologies orientées objet dans une entreprise. Celles-ci n'ont pas de répercussions sur les pratiques des gestionnaires de projets et des analystes.

Questions«Vrai ou Faux»

- **R3:**

C'est **faux** parce qu'au contraire les technologies orientées objet bouleversent les pratiques des entreprises à tous les niveaux.

Les gestionnaires ne peuvent plus savoir quand et combien cela va coûter.

Les analystes perdent la maîtrise sur les outils d'analyse, de modélisation, de programmation, de documentation...

De fait, tous les intervenants doivent apprivoiser cette transformation et cela ne doit pas être pris à la légère.

Questions«Vrai ou Faux»

- **Q4:**
- L'analyse orientée objet est une technique difficile parce qu'on y spécifie des entités qui n'ont qu'un lien très éloigné avec la réalité.

- **R4**

C'est **faux**. On mentionne plutôt que l'approche orientée objet permet de réduire le fossé sémantique entre la réalité et ce qui est implanté, le code. La maintenance et l'évolution doit en principe en être facilité.

- **Q5**

L'intérêt des langages orientés objets est qu'ils forcent le programmeur à avoir une approche orientée objet.

- **R5**

C'est **faux**. C'est d'abord une façon de découper des problèmes qui n'a rien à voir avec le langage de programmation.

De fait, on découpe selon des principes orientés objet et on pourrait en partie implanter cette analyse avec un langage traditionnel comme le langage C.

- Par contre, un langage orienté objet comme le C++, Java, Smalltalk offre un support supplémentaire. On peut dire qu'on a plus de vocabulaire pour exprimer nos idées...

- **Q6**

Une classe doit représenter un groupe d'objets ayant un comportement commun et une structure commune.

- R6
- C'est **vrai** puisque la classe sert à spécifier comment les objets de ce type devront se comporter de leur création à leur destruction.

- Q7:
- L'approche orientée objets n'est pas neuve en soi; elle ne fait que se servir de trois concepts connus: l'abstraction, la hiérarchie et la décomposition.

- R7
- C'est **vrai**. Ces trois façons d'appréhender un problème existent depuis fort longtemps.

- Q8:
- Une classe a trois composantes: un état, un comportement et une identité unique.

- R8
- C'est **faux**. C'est simplement une astuce : à la place de classe on aurait dû avoir objet.

- Q9:
- Un des principes de base de l'orienté objet serait de dire qu'on est jamais mieux servi que par soi-même.

- R9:
- C'est **faux**. Un grand principe de base est la confiance dans le travail des autres ou délégation. Il faut se fier à ce qui est promis par l'interface de la classe et non aller voir comment c'est implanté.

- Q10
- La modélisation visuelle, bien qu'elle soit toujours un peu ambiguë, permet tout de même de mieux comprendre les problèmes à l'étude car un dessin vaut mille mots.

- R10
- C'est **faux**. La modélisation visuelle n'est pas ambiguë. Le langage UML a permis de spécifier sans ambiguïté un design orienté objet sous forme visuelle.

EX01

- Quel est le rôle de l' ' opérateur de résolution de portée? (::)

R.Exo1

- Si une fonction membre d'une classe est implémentée à l'extérieur de la définition de cette classe, le nom de la fonction doit être précédé par le nom de la classe et par l'opérateur de résolution de portée ::

- Puisque différentes classes peuvent avoir des noms de membres identiques, cela permet de lier la fonction à une classe en particulier sans qu'il y ait d'ambiguïté.
- Exemple: `Date :: affichage();`

Exo2

- Comparez les notions de struct et de class en C++ et soulignez - en les contrastes

R.Exo2

struct :

- On ne peut pas afficher une struct directement
- On ne peut pas comparer deux struct directement
- On peut accéder aux données de n'importe où

class :

- Un objet peut être affiché directement en surchargeant l'opérateur <<
- On peut surcharger les différents opérateurs de comparaison afin de pouvoir comparer deux objets
- On peut limiter l'accès aux données à l'aide des identificateurs d'accès aux membres (private, public ou protected)

struct :

- Chaque fonction manipulant les données d'une struct doit recevoir cette struct en paramètre
- Une struct n'est pas forcément initialisée avant son utilisation

class :

- Les fonctions membres ont directement accès aux données d'un objet
- Un objet est forcément instancié, et donc initialisé, lors de sa déclaration

Exo3

- Fournissez un constructeur capable d'utiliser l'heure courante à partir de la fonction `time()`, déclarée dans l'en - tête de la bibliothèque standard `time.h` du C, pour initialiser un objet de la classe `Temps`.

R.Exo3

- Au début du fichier ajouter :
- `#include "time.h"`
- Le nouveau constructeur :

// Fonction constructeur utilisant l'heure courante à partir de la fonction time.

Temps::Temps()

{

time_t secondes;

secondes = time(NULL); // secondes depuis le 1er janvier 1970 à 00:00 selon
// l'heure de Greenwich

heure = secondes / 3600; // nombre d'heures depuis 1970

heure = heure % 24; // nombre d'heures aujourd'hui

heure = heure - 5; // nombre d'heures aujourd'hui à Québec (fuseau horaire)

minute = secondes / 60; // nombre de minutes depuis 1970

minute = minute % (24 * 60); // nombre de minutes aujourd'hui

minute = minute % 60; // nombre de minutes dans la dernière heure

seconde = secondes % (24 * 3600); // nombre de secondes aujourd'hui

seconde = seconde % 60; // nombre de secondes dans la dernière heure

seconde = seconde % 60; // nombre de secondes dans la dernière minute

}

Exo4

- Créez une classe appelée Complexe capable d'effectuer des opérations arithmétiques avec des nombres complexes.
- Rappel: Les nombres complexes possèdent la forme suivante: partie réelle + partie imaginaire* i où $i^2 = -1$
- Exemple: $a + b*i$

R.Exo4: le Fichier complexe.h

```
#ifndef COMPLEXE_H                // Directives de précompilation qui empêchent
#define COMPLEXE_H                // de multiples inclusions du fichier d'en-tête.
#include <iostream>
using namespace std;

class Complexe
{
public:
    Complexe(double r = 0.0, double i = 0.0);           // constructeur.

    Complexe addition(const Complexe & );               // fonction d'addition.
    Complexe soustraction(const Complexe & );           // fonction de soustraction.
    Complexe multiplication(const Complexe & );         // fonction de multiplication.
    Complexe conjugué();                                 // fonction qui donne le conjugué.
    void afficherComplexe();                             // fonction d'affichage.

private:
    double reel;
    double imaginaire;
};
#endif
```


le Fichier complexe.cpp

```
#include <iostream>
#include <stdlib.h>

using std::cin;
using std::cout;

#include "complexe.h"

// constructeur.
Complexe::Complexe(double r, double i)
{
    reel = r;
    imaginaire = i;
}
```

le Fichier complexe.cpp (suite....)

```
// les fonctions
Complexe Complexe::addition(const Complexe &c2)
{
    return Complexe(reel + c2.reel, imaginaire + c2.imaginaire);
}
Complexe Complexe::soustraction( const Complexe &c2)
{
    return Complexe(reel - c2.reel, imaginaire - c2.imaginaire);
}
void Complexe::afficherComplexe()
{
    cout << reel << " + " << imaginaire << " *i ";
}
```

// les fonctions (suite)

```
Complexe Complexe::multiplication( const Complexe &c2)
{
    return Complexe(reel * c2.reel - imaginaire * c2.imaginaire,
                    reel * c2.imaginaire + imaginaire * c2.reel);
}
```

```
Complexe Complexe::conjugue()
{
    return Complexe(reel, -imaginaire);
}
```

le Fichier principal.cpp

```
#include <iostream>
using namespace std;
int main()
{
    double r;
    double i;

    cout << "Entrez la partie reelle : ";
    cin >> r;
    cout << "Entrez la partie imaginaire : ";
    cin >> i;
    complexe c1(r,i);
    cout << "Entrez la partie reelle : ";
    cin >> r;
    cout << "Entrez la partie imaginaire : ";
    cin >> i;
    complexe c2(r,i);
```

```
    Complexe c3 = c1.addition(c2);
    Complexe c4 = c3;
    c4.conjugué();

    c1.afficherComplexe();
    cout << " + ";
    c2.afficherComplexe();
    cout << " = ";
    c3.afficherComplexe();
    cout << endl;
    cout << " Son conjugué est: ";
    c4.afficherComplexe();

    return 0;
}
```

Exo5

- Une définition de classe Temps correcte peut-elle inclure les deux constructeurs suivants?
- **Temps (int h = 0, int m = 0, int s = 0);**
- **Temps();**

R.Exo5

- Non. Si le constructeur par défaut est utilisé, il y aura confusion entre les deux constructeurs.

Exo6

- Que se passe t-il lorsqu'on spécifie un type de retour, incluant void, pour un constructeur ou un destructeur ?

R.Exo6

- Il y aura une erreur de syntaxe à la compilation.